Worked for 24s

I think this result is pointing at something more important than “procedural training needs more data.”

The v2.1.2 student appears to be showing **capacity reallocation**: broad reasoning training made the model much more generally competent, but some of the original specialized behaviors were partially displaced. That is exactly the kind of tradeoff we should now study deliberately rather than continuing to optimize one metric at a time.

There is one important issue in the current v2.1.2 report, though: the selector says `stage_b2_mixed` wins with reasoning rising from **0.003 → 0.437**, hard reasoning to **0.471**, and abstention on the new reasoning set to **0.967**. GENERAL_REASONING_V212_SELECTION But in the JSON, all of the original Phase 9 preservation fields—procedure correctness, supported correctness, faithfulness, abstention, conflict handling, and social usability—are `null`. general_reasoning_v212_selection So the reported `preservation_cost = 0` is not trustworthy: it means **those original evaluations were missing**, not that there was no regression.

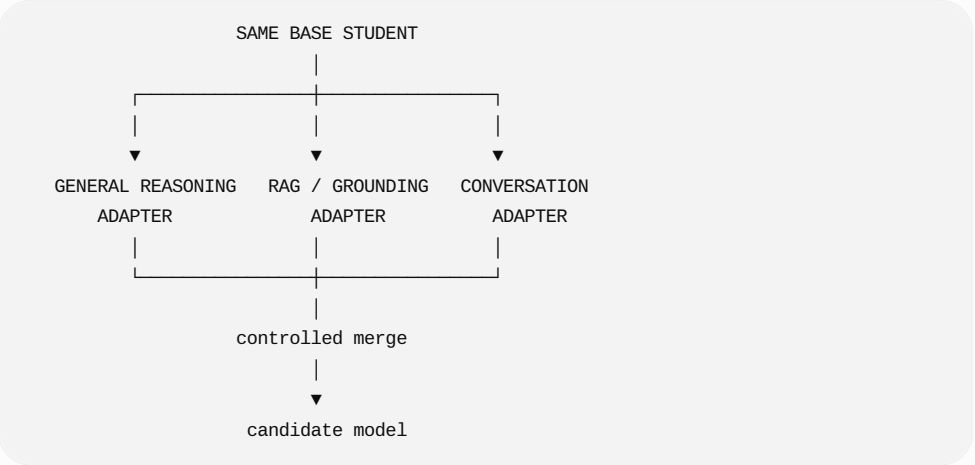
That should be fixed before we use the selector to make another training decision.

The direction I would pursue

I would stop treating this as a single-adapter recovery problem and turn it into a **capability-composition experiment**.

The original student is valuable because it already contains unusually strong specialized behavior. The broad-reasoning adapter is valuable because it appears to unlock capabilities that task-specific procedural training barely moved.

So instead of forcing one fine-tuning run to learn everything, I would create:



That is very close to the experiment I think your results are asking us to run.

Why I like your broader dataset idea

Your category list is good:

> General / Research / Writing / Communication / Translation / Coding / DevOps / Systems / Math / Data Analysis / Science / Engineering / Education /

> Business / Operations / Finance / Legal / Health / Consumer / Travel / Home / Creative / Media / Personal / Coaching / Tools / Agents

But I would make one important distinction.

I would **not primarily teach domain facts**.

For a tiny model, the dataset should teach reusable operations inside those domains:

```
compare
rank
filter
infer
calculate
diagnose
sequence
track state
resolve constraints
apply rules
spot conflicts
identify missing information
choose evidence
summarize
translate intent
follow procedures
verify conditions
```

Then each domain provides a different distribution of reasoning situations.

For example:

```
Finance
→ percentage changes
→ budget constraints
→ comparison
→ insufficient evidence

Coding
→ trace program state
→ identify bug cause
→ dependency reasoning
→ choose next debugging action

Legal
→ rule + exception
→ applicability
→ missing jurisdiction/context
→ safe uncertainty

Health
→ symptom information extraction
→ contraindication awareness
→ uncertainty / escalation
→ no unsupported diagnosis

DevOps
→ dependency chains
→ service state
→ logs + evidence
→ rollback conditions
```

That gives the tiny model **procedural mental machinery**, not a giant internal fact database.

That also fits the original blueprint, which explicitly defines this as a specialized short-response model rather than a replacement for a large universal reasoning model. Short_Response_Model_Research_a...

I would run four experiments next

A. Broad-domain reasoning adapter

Train one fresh adapter from the **original student**, not from the v2.1.2 adapter.

Something like:

```
Original student
      │
      └── General-Reasoning-v2 adapter
          20–30k high-quality examples
          many domains
          reasoning-first
          short final outputs
```

Do not sequentially keep adding adapters yet.

Sequential fine-tuning makes attribution very hard:

```
original
→ procedure FT
→ general reasoning FT
→ preservation FT
→ ...
```

Each stage can overwrite the previous one.

Parallel adapters give us clean causal experiments.

B. Preserve the original student as an anchor adapter/checkpoint

Keep the original behavior as the reference distribution.

Then train general reasoning with an explicit preservation term.

Conceptually:

$$L = L_{\text{general}} + \lambda_1 L_{\text{specialized}} + \lambda_2 KL(p_{\text{new}} || p_{\text{original}})$$

The third term is particularly interesting.

On preservation examples:

```
original student logits
        │
        ▼
  distribution target

new student logits
        │
        ▼
     KL penalty
```

So we tell training:

> Learn the new reasoning skill, but don't move unnecessarily on behaviors the original student already knows.

That is much more principled than merely replaying old examples.

It is essentially **behavioral anchoring**.

I think this should become one of our main experiments.

C. Train specialized adapters separately and merge them

This is where your LoRA idea becomes especially interesting.

Suppose we produce:

```
Δgeneral
Δrag
Δprocedure
Δsocial
Δabstention
```

Instead of stacking all their training sequentially, evaluate weighted combinations:

$$\Delta = \alpha_g \Delta_g + \alpha_r \Delta_r + \alpha_p \Delta_p + \alpha_s \Delta_s + \alpha_a \Delta_a$$

For example:

```
General reasoning      0.50
Original grounding     0.25
Procedural             0.10
Abstention             0.10
Social                 0.05
```

Then evaluate.

Change weights:

```
0.40 / 0.30 / 0.10 / 0.15 / 0.05
0.35 / 0.35 / 0.10 / 0.15 / 0.05
...
```

This is inexpensive compared with retraining because merging adapter deltas does not require another full training run.

The idea is closely related to the model-soup principle: independently fine-tuned descendants of a shared initialization can sometimes be weight-averaged into a model that is better and more robust than the best individual run, without increasing inference cost. Model soup

For LoRA deltas specifically we still have to test compatibility empirically; I would **not assume arbitrary adapters merge cleanly**.

But your setup is unusually favorable because all adapters would come from the exact same student initialization.

The experiment I find even more interesting: layer-separated adapters

This project already has unusually strong layer-analysis infrastructure.

So I would test whether **different capabilities prefer different layers**.

For example:

```
Layer 0 ──────────────────────────
Layer 1
Layer 2
...
Layer 7        RAG / binding LoRA
Layer 8        RAG / binding LoRA
Layer 9
Layer 10       general reasoning LoRA
Layer 11       general reasoning LoRA
Layer 12       general reasoning LoRA
Layer 13
Layer 14       conversation LoRA
...
```

That gives us:

> **capability specialization in parameter space**

instead of making five objectives fight over the same LoRA matrices.

There is real motivation for trying this. The IST work reports that PEFT adaptation is highly non-uniform across layers and that updating important subsets can outperform uniformly updating every layer.
Layer-wise Importance Matters L… The single-layer RL study likewise finds that adaptation gains can be concentrated in a small subset of layers, with strong layers often clustering around the middle of the stack.
IS ONE LAYER ENOUGH TRAINING A …

Those papers do **not** prove that your reasoning and RAG skills occupy separate layers.

But your repository is now in a perfect position to test that hypothesis.

I would actually prioritize this over a huge LoRA-rank sweep.

So the next experiment becomes a capability matrix

Instead of:

```
Did Real-RAG procedure improve?
```

we evaluate:

Candidate	General reason	RAG supported	RAG procedure	Faithfulness	Abstention	Social
Original student						

Candidate	General reason	RAG supported	RAG procedure	Faithfulness	Abstention	Social
General adapter						
RAG adapter						
Procedure adapter						
General + RAG merge						
General + preservation KL						
Layer-separated adapters						
Greedy adapter soup						

Then we look at the **Pareto frontier**, not a single score.

That is important because your findings are already showing that there may not be one scalar notion of “better.”

I would also change the datasets substantially

For the next broad corpus, I would target perhaps **30k–50k examples**, but make them more heterogeneous than v2.1.2.

Not simply:

```
30,000 versions of arithmetic and ranking
```

Instead:

```
General reasoning
Research synthesis
Technical explanation
Writing transformation
Communication intent
Translation
Coding reasoning
Debugging
DevOps
Systems reasoning
Math
Statistics/data analysis
Science
Engineering
Education
Business
Operations
Finance
```

```
Legal reasoning
Health reasoning
Consumer decisions
Travel planning
Home troubleshooting
Creative constraints
Media reasoning
Personal planning
Coaching
Tool use
Agent state / workflow
RAG evidence reasoning
Memory reconstruction
```

And within each domain:

```
easy
moderate
hard
ambiguous
conflicting evidence
insufficient evidence
distractor evidence
multi-step
counterfactual
exception
boundary condition
```

That distribution is more important than raw dataset size.

One thing I would not do

I would not create one enormous adapter and simply keep training longer.

Your results are already warning us about what happens when we do that:

```
specialist capability
        ↓
large new training distribution
        ↓
parameter movement
        ↓
new ability appears
        +
some old ability moves
```

That is ordinary interference / forgetting.

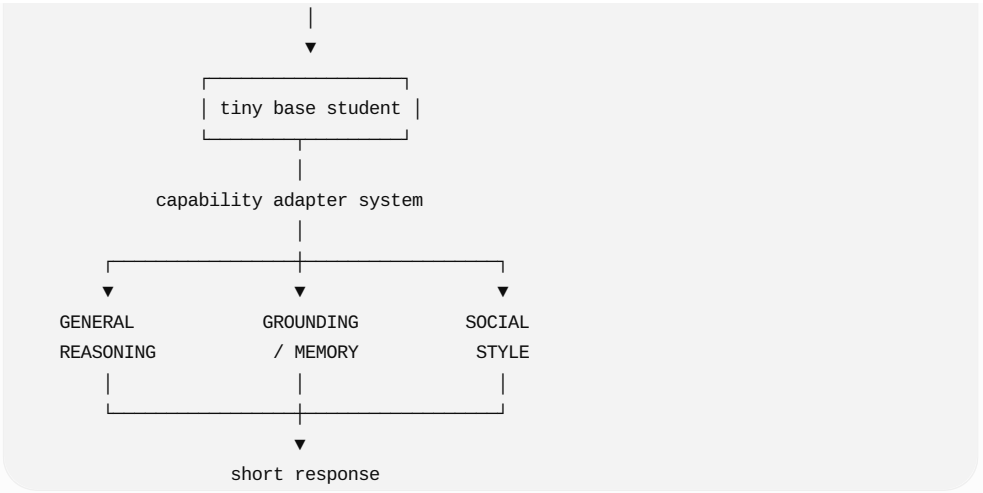
More data could make the general model better while **further erasing the specialist**.

So I would treat the original student as a valuable frozen asset.

The architecture I would aim toward

Long term, I think your short-response model may end up looking like this:

```
                    RAG
                     |
                     ▼
              evidence selector
```



During research, adapters can remain separate.

Later we ask whether we can collapse them into:

```
one merged adapter
```

or even:

```
one merged dense student
```

without losing the complementary skills.

If we cannot, an extremely lightweight adapter router is also viable.

The most important immediate action

Before the next training run, I would fix **v2.1.2 evaluation plumbing**.

Your current selection JSON contains:

```
supported_answer_correctness: null
procedure_correctness: null
evidence_faithfulness: null
correct_abstention_rate: null
conflict_safe_handling_rate: null
social_usable_rate: null
```

for every candidate. `general_reasoning_v212_selection`

Therefore the current reported:

```
preservation_cost = 0
```

does **not** demonstrate preservation.

It means the preservation evidence never reached the selector.

That is the first thing I would correct before v2.1.3.

My preferred next version

I would make the next repository **v2.1.3 — Multi-Domain Capability Composition**, with four distinct tracks:

```
TRACK A
Expanded 30k–50k multi-domain reasoning curriculum

TRACK B
General reasoning + KL anchor to original student

TRACK C
Parallel specialist LoRA adapters
general / grounding / procedure / abstention / social

TRACK D
Adapter composition research
weighted merge
greedy merge
layer-separated merge
```

And every track is evaluated against the **frozen original student**.

That would answer a much deeper question than the earlier Phase 9 work:

> **Is this tiny student's capacity fundamentally too small, or can specialized abilities coexist if we stop forcing every skill through the same parameter updates?**

I think that is now the most valuable question in this project.

Your v2.1.2 result makes me **more optimistic**, not less. Going from essentially **0.3% → 43.7% general-reasoning correctness** and **1.4% → 47.1% hard reasoning** means this architecture still has a surprisingly large amount of learnable capacity. `GENERAL_REASONING_V212_SELECTION` The research problem has shifted from *"can it learn?"* to **"how do we allocate and combine what it learns without overwriting what it already does well?"**

Sources